Problem H. Hidden Permutation

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

This is a communication problem. Your program will be run twice for each test, in two independent processes.

First Run

You are given a permutation $p = (p_1, p_2, \ldots, p_n)$ of $1, 2, \ldots, n$.

Output an $n \times n$ matrix A such that each row is a permutation of $1, 2, \ldots, n$ and all rows are pairwise distinct.

Second Run

Before the second run, the judge randomly shuffles the rows of A to obtain a matrix B. The order of the elements within each row remains unchanged. You are given B and must recover the original permutation p from the first run.

Your program must recover p for every possible ordering of the rows. No process state is preserved between the two runs; the shuffled matrix is the only information passed from the first run to the second. The time and memory limits apply to each run separately.

Input

The first line contains a string `op` (either `first` or `second`) and an integer n ($3 \le n \le 2000$), indicating the run and the size of the permutation. The value of n is the same in both runs.

If `op` is `first`, the second line contains n integers $p_1, p_2, \ldots, p_n$ ($1 \le p_i \le n$), a permutation of $1, 2, \ldots, n$.

If `op` is `second`, each of the next n lines contains n integers, describing one row of B. This is the matrix output in the first run with only its rows randomly reordered.

There is one test case per run.

Output

If `op` is `first`, output n lines, each containing n space-separated integers, describing the matrix A. Each row must contain every integer from 1 to n exactly once, and the n rows must be pairwise distinct.

If `op` is `second`, output n space-separated integers on one line: the original permutation p from the first run, in its original order.

Examples

standard input	standard output
first 3 2 3 1	2 3 1 1 2 3 3 2 1
second 3 3 2 1 2 3 1 1 2 3	2 3 1

Note

The two examples show the first and second runs of one possible solution on the same test.

In the first run, the input permutation is $(2, 3, 1)$, and the program outputs a 3×3 matrix whose rows are pairwise distinct permutations of $1, 2, 3$.

Before the second run, the judge places the third, first, and second rows of this matrix in that order. The elements within each row remain unchanged. From the resulting matrix, the program must recover and output $(2, 3, 1)$. The row order used during judging may differ from the example.